

Tema 4.2. Herencia

1. En orientación a objetos, ¿qué es la herencia y su relación con "A es-un B"?. Explica las dos implicaciones principales: (1) **compatibilidad de tipos y (2) **herencia de estado y comportamiento**. Pon un ejemplo en Java muy sencillo, donde un `Soldado` tiene un `nombre` (privado) y un método `saludar()` que muestra su nombre. Hay dos subtipos: un `Artillero`, que es capaz de disparar cohetes y un `Zapador` que pone minas, ambos heredan el atributo `nombre` y la capacidad de saludar. Además, y de forma específica, el `artillero` tiene un número de cohetes y el `zapador` un número de minas, accesibles mediante "getters" específicos. Respecto a la compatibilidad de tipos, aprovechémosla: crea un array de `Soldado`, mete varios de distinto tipo (son todos compatibles con `Soldado`). Recórrela y que todos te saluden.**

(1)-> ¿qué es la **herencia** y su relación con "A es-un B"?

La herencia es un mecanismo fundamental de la Programación Orientada a Objetos (POO) que permite a una clase nueva (clase hija/subclase) adquirir las características (atributos y métodos) de una clase existente (clase padre/superclase).

La relación "A es-un B" (por ejemplo, "Un Artillero es un Soldado") es la regla de oro para saber si debemos usar herencia. Si la frase tiene sentido lógico, la herencia es el camino correcto. Se diferencia conceptualmente de la composición ("A tiene un B"), donde una clase simplemente guarda una instancia de otra como atributo (ej. Un Soldado tiene un Rifle).

(2)->

Compatibilidad de tipos

Significa que un objeto de la subclase puede ser tratado como si fuera del tipo de la superclase. Como un Artillero es un Soldado, puedes almacenar un Artillero en una variable de tipo Soldado. Esto permite crear código genérico (como tu método pasarRevista) que funciona con cualquier subtipo actual o futuro sin necesidad de modificarse.

Herencia de estado y comportamiento

La subclase no necesita volver a definir los atributos (estado) ni los métodos (comportamiento) que ya existen en la superclase. Los hereda automáticamente. Un Zapador ya sabe saludar() y tiene un nombre simplemente por heredar de Soldado, permitiendo la reutilización de código.

***CLASE

Composición -> "Tiene un/tiene varios" vs. Herencia-> "es un"

1-> Compatibilidad de tipos

Soldado s= new Artillero("Pepe");

2-> Herencia de estado y comportamiento

Todos los atributos de estado estan en Artillero y todos los metodos del comportamiento está en Artillero ç

```

public class Soldado {
    private String nombre;

    public Soldado (String nombre){
        this.nombre= nombre;
    }

    public void saludar(){
        sout ("Le saluda el soldado"+this.nombre);
    }
    // Getter para que las subclases puedan usar el nombre si lo necesitan
    public String getNombre() {
        return nombre;
    }
}

```

```

public class Artillero extends Soldado { //establecemos la herencia
    public int numCohetes;

    public Artillero(String nombre,int numCohetes){
        super(nombre);
        this.numCohetes=numCohetes;
    }

    public int getnumCohetes(){
        return this.numCohetes;
    }
}

```

```

public class Zapador extends Soldado { //establecemos la herencia
    public int numMinas;

    public Zapador(String nombre, int numMinas){
        super (nombre);
        this.numMinas=numMinas;
    }

    public int getnumCohetes(){
        return this.numMinas;
    }
}

```

```

public class PruebaHerencia{

```

```

    public static void pasarRevista(Soldado[] soldados){
        for (Soldado soldado:soldados){
            soldado.saludar();
        }
    }
    public static void main (String [] args){
        Soldado [] soldadosVarios = new Soldado [3];
        soldadosVarios [0]= new Artillero ("Juan", 3);
        soldadosVarios[1] = new Artillero ("Ana", 1);
        soldadosVarios [2]= new Zapador ("Pepe", 10);

        pasarRevista(soldadosVarios); //metodo que se ha quedado cerrado independientemente
    }
}

```

2. Al crear los soldados concretos, ¿cuántos constructores se ejecutan y en qué orden? ¿Qué significa `super` dentro de un constructor? Si la clase base no tiene visible el constructor sin parámetros, ¿debo llamar a `super` siempre?

(1)-> ¿Cuántos constructores se ejecutan y en qué orden?

Como bien has dicho, se ejecuta un constructor por cada clase de la jerarquía, moviéndose de arriba a abajo (desde la superclase más alta hasta la subclase concreta).

Cuando creas un Artillero:

Se inicia la llamada al constructor de Artillero.

Este, antes de hacer nada, delega hacia arriba llamando al constructor de Soldado.

Soldado (que implícitamente hereda de la clase cósmica `Object` de Java) delega en `Object`.

Se ejecuta el cuerpo de `Object`, luego el cuerpo de `Soldado` y, finalmente, se ejecuta el cuerpo de `Artillero`.

Regla de oro: No puedes construir un hijo sin haber construido primero los cimientos de su padre. El estado heredado debe inicializarse antes de que la subclase añada sus propias particularidades.

(2)-> ¿Qué significa `super` dentro de un constructor?

Dentro de un constructor, la palabra clave `super(...)` es una llamada explícita al constructor de la

superclase. Funciona como una lanzadera de datos: le pasa los argumentos necesarios al padre para que este pueda inicializar sus atributos privados.

En tu código tenías una pequeña coma suelta en el `super(nombre, )`. La sintaxis limpia y exacta es simplemente pasarle la variable:

```
public Artillero(int numCohetes, String nombre) {  
    super(nombre); // Llama al constructor de Soldado pasándole el nombre  
    this.numCohetes = numCohetes; // Luego inicializa lo propio de Artillero  
}
```

(3)-> Si la clase base no tiene visible el constructor sin parámetros, ¿debo llamar a `super` siempre? Si no escribes `super(...)` manualmente, Java intenta insertar de forma automática y transparente un `super()` (sin parámetros) en la primera línea de tu constructor.

Si la clase padre tiene un constructor sin parámetros (o no tiene ningún constructor definido): El `super()` invisible de Java funciona sin problemas.

Si la clase padre TIENE constructores con parámetros (como nuestro `Soldado(String nombre)`) y NO has definido un constructor vacío: El compilador de Java no encuentra un constructor por defecto al que invocar automáticamente y lanzará un error de compilación. Te obligará a escribir `super(nombre)` explícitamente.

Un detalle crucial sobre `super`

La llamada a `super(...)` debe ser siempre la primera línea de código dentro del constructor de la subclase. Si intentas hacer un `this.numCohetes = numCohetes;` antes de `super(nombre);`, Java se negará a compilar. Primero el padre, luego el hijo.

***CLASE

Se ejecutan un constructor por cada clase de la jerarquía y se ejecutan de arriba a abajo. `Super` indica como debe invocarse el superconstructor desde la subclase

SI, si tu clase base a solo tiene el constructor con algun parametro

```

public class Artillero extends Soldado { //establecemos la herencia

    public int numCohetes;

    public Artillero(int numCohetes, String nombre){
        //Nos obliga a llamar al superConstructor
        super(nombre);
        this.numCohetes=numCohetes;
    }

    public int getnumCohetes(){
        return this.numCohetes;
    }
}

```

3. Respecto a los objetos de subclases en memoria, los atributos privados de la superclase, ¿forman parte de una instancia de la subclase en memoria? En caso afirmativo ¿implica que se puedan usar desde el código de la subclase? Explícalo con el ejemplo de Soldado y alguna de sus subclases.

Esta es una de las preguntas más brillantes y que más confusión genera en la programación orientada a objetos. La respuesta corta es: Sí, forman parte del objeto en memoria, pero No, no puedes usarlos directamente desde el código de la subclase.

Aunque parezca una contradicción, tiene todo el sentido del mundo cuando miras cómo se organiza la memoria. Vamos a destriparlo.

(1)->En la memoria: ¿El atributo privado forma parte del objeto?

Sí, absolutamente. Cuando tú creas un objeto de una subclase (por ejemplo, un Artillero), Java reserva un único bloque de memoria para esa instancia. Dentro de ese bloque está todo: tanto los atributos que hereda de la superclase (nombre) como los suyos propios (numCohetes).

Si el atributo nombre no estuviera en la memoria del objeto Artillero, cuando llamaras al método saludar(), la máquina virtual de Java no sabría qué nombre imprimir. El estado existe y ocupa espacio en el objeto.

(2)-> En el código: ¿Se puede usar desde la subclase?

No, directamente no. Aquí es donde entra en juego el concepto de encapsulamiento y los modificadores de acceso (private).

Que un atributo sea private significa que solo las líneas de código que estén escritas dentro de la clase Soldado tienen permiso para leerlo o modificarlo. La clase Artillero, por mucho que sea su "hija", es una clase distinta, así que el compilador le tiene prohibido el acceso directo.

```
public class Soldado {
    private String nombre; // En memoria SÍ existe para todos, pero está "blindado"

    public Soldado(String nombre) {
        this.nombre = nombre;
    }
}

public class Artillero extends Soldado {
    private int numCohetes;

    public Artillero(String nombre, int numCohetes) {
        super(nombre);
        this.numCohetes = numCohetes;
    }

    public void mostrarDatos() {
        // INTENTO 1: Acceso directo
        // ERROR DE COMPILACIÓN: "nombre has private access in Soldado"
        System.out.println("Me llamo: " + this.nombre);

        // INTENTO 2: Acceso indirecto (La forma correcta)
        // FUNCIONA: Usamos un método público que el padre nos presta para mirar dentro
        System.out.println("Me llamo: " + getNombre());
    }
}
```

***CLASE

(1)-> ¿forman parte de una instancia de la subclase en memoria?

Sí.

(2)-> ¿Los atributos privados de la superclase están en la subclase?

Sí.

Cuando creas un objeto de una subclase, en memoria se construye un único objeto que contiene:

La parte de la superclase

La parte de la subclase

Es decir, la subclase hereda la estructura en memoria, aunque no todo sea accesible.

EJEMPLO:

```
class Soldado {
    private String nombre;
    private int edad;

    public Soldado(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}

class SoldadoRaso extends Soldado {
    private String rango;

    public SoldadoRaso(String nombre, int edad, String rango) {
        super(nombre, edad);
        this.rango = rango;
    }
}
```

(3)-> ¿Se pueden usar directamente desde la subclase?

No, no se pueden acceder directamente.

El modificador private significa:

Solo accesible dentro de la propia clase Soldado

Ni siquiera las subclases pueden acceder directamente

```
class SoldadoRaso extends Soldado {

    public void mostrar() {
        System.out.println(nombre); // ERROR
    }
}
```


4. ¿Qué implica en términos de **extensibilidad** de código el hecho de que sean compatibles a nivel de tipos? Ilustra esto añadiendo un nuevo tipo de Soldado y demostrando que el código para pedir el saludo a todos los soldados no se modifica.

(1)->¿Qué implica en términos de **extensibilidad** de código el hecho de que sean compatibles a nivel de tipos?

Cuando dos tipos son compatibles a nivel de tipos, significa que el código que trabaja con una abstracción (por ejemplo, una interfaz o tipo base) puede aceptar nuevos tipos sin modificarse. En términos de extensibilidad, esto es clave: puedes añadir comportamiento nuevo sin tocar el código existente, siempre que respetes el contrato del tipo.

***CLASE

¿Qué implica para la extensibilidad?

Que puedes:

Añadir nuevas subclases de Soldado

Sin modificar el código existente que trabaja con Soldado

Esto es exactamente el principio “abierto/cerrado”:

Abierto a extensión (puedes añadir nuevas clases)

Cerrado a modificación (no tienes que tocar código ya hecho)

```

abstract class Soldado {
    protected String nombre;

    public Soldado(String nombre) {
        this.nombre = nombre;
    }

    public abstract void saludar();
}

class Artillero extends Soldado {
    private int nivel;

    public Artillero(String nombre, int nivel) {
        super(nombre);
        this.nivel = nivel;
    }

    public void saludar() {
        System.out.println("Soy artillero " + nombre);
    }
}

class FrancoTirador extends Soldado {
    private int nivel;

    public FrancoTirador(String nombre, int nivel) {
        super(nombre);
        this.nivel = nivel;
    }

    public void saludar() {
        System.out.println("Soy artFrancoTirador" + nombre);
    }
}

class Medico extends Soldado {
    private int experiencia;

    public Medico(String nombre, int experiencia) {
        super(nombre);
        this.experiencia = experiencia;
    }
}

```

```

    public void saludar() {
        System.out.println("Soy médico " + nombre);
    }
}

public class PruebaHerencia {

    public static void pasarRevista(Soldado[] soldados){
        for (Soldado soldado : soldados){
            soldado.saludar(); // polimorfismo
        }
    }

    public static void main(String[] args){
        Soldado[] soldadosVarios = new Soldado[4];

        soldadosVarios[0] = new Artillero("Juan", 3);
        soldadosVarios[1] = new Artillero("Ana", 1);
        soldadosVarios[2] = new Medico("Pepe", 10);
        soldadosVarios[3] = new Francotirador("Luis", 95); // NUEVO

        pasarRevista(soldadosVarios);
    }
}

```

5. En Java, cuando trabajo con referencias y herencia. ¿Puedo tener una referencia del supertipo que apunte a objetos reales de un subtipo? ¿Puedo invocar con la referencia del supertipo a métodos públicos del subtipo? ¿En qué consiste el "upcasting" y el "downcasting"? ¿Qué es el instanceof ? Pon un ejemplo de recorrido de un array de Soldado , comprobando que, si el objeto real es un Artillero , solicite el número de cohetes que tiene y los imprima.

(1)-> ¿Puedo tener una referencia del supertipo que apunte a objetos reales de un subtipo?

Sí

```
Soldado s = new Artillero("Juan", 3);
```

-s es del tipo Soldado (referencia)

-Pero el objeto real en memoria es un Artillero

Esto es totalmente válido.

(2)-> ¿Puedo invocar con la referencia del supertipo a métodos públicos del subtipo?

Depende

Solo puedes llamar a métodos definidos en el supertipo

```
s.saludar(); // OK (está en Soldado)
```

No puedes llamar directamente métodos específicos del subtipo

```
s.getCohetes(); // ERROR si no está en Soldado
```

Aunque el objeto real sea Artillero, el compilador solo mira el tipo de la referencia.

(3)->¿En qué consiste el **"upcasting"** y el **"downcasting"**?

"Upcasting"(conversión hacia arriba)

Es cuando conviertes un subtipo a supertipo:

```
Soldado s = new Artillero("Juan", 3);
```

"Downcasting" (conversión hacia abajo)

Es cuando conviertes un supertipo a subtipo:

```
Artillero a = (Artillero) s;
```

(4)->¿Qué es el `instanceof` ?

Sirve para comprobar el tipo real del objeto en tiempo de ejecución:

```
if (s instanceof Artillero) {  
    // es seguro hacer downcast  
}
```

***CLASE

```
public static void main(String[] args){
    Soldado[] soldadosVarios = new Soldado[4];

    soldadosVarios[0] = new Artillero("Juan", 3);
    soldadosVarios[1] = new Artillero("Ana", 1);
    soldadosVarios[2] = new Medico("Pepe", 10);
    soldadosVarios[3] = new Francotirador("Luis", 95); // NUEVO

    pasarRevista(soldadosVarios);
    Soldado soldado = new Artillero ("juan", 10); //upcasting, es automatico (implicito)

    if (soldado instanceof Artillero){ //downcasting automatico
        Artillero comoArtillero= (Artillero) soldado ;//downcasting, explicitco (eso que va
        int cohetes = comoArtillero.getnumCohetes());
        sout ("cohetes"+cohetes);
    }

    if (soldado instanceof Medico) {
        Medico medico = (Medico)soldado; //downcasting, explicitco (eso que va entre parentesis)
    }

}
```

6. Respecto a la ocultación de información y herencia, ¿qué significa acceso "protegido" de métodos y/o atributos? ¿Cómo se implementa en Java? Pon un ejemplo de uso de en la clase Soldado para que su nombre sea protegido y pueda usarse en el método de poner bombas del Zapador .

(1)-> ¿Qué significa acceso "protegido" de métodos y/o atributos?

En Java, el modificador protected es una forma intermedia de control de acceso dentro del principio de ocultación de información (encapsulación).

¿Qué significa protected?

Un atributo o método protected es accesible desde:

Sí puede acceder:

La propia clase

Subclases (aunque estén en otro paquete)

Clases del mismo paquete

No puede acceder:

Clases externas que no heredan ni están en el mismo paquete

(2)->¿Cómo se implementa en Java?

```
protected tipo nombre;          //0 uno u otro  
protected void metodo() { }
```

***CLASE

```

class Soldado {
    protected String nombre; // ahora es accesible por las subclases

    public Soldado(String nombre) {
        this.nombre = nombre;
    }

    public void saludar() {
        System.out.println("Soy el soldado " + nombre);
    }
}

class Zapador extends Soldado {

    public Zapador(String nombre) {
        super(nombre);
    }

    public void ponerBomba() {
        // acceso directo al atributo protegido
        System.out.println(nombre + " está colocando una bomba");
    }
}

public class Prueba {
    public static void main(String[] args) {
        Zapador z = new Zapador("Carlos");
        z.ponerBomba();
    }
}

```

7. En los lenguajes orientados a objetos ¿hay una **clase base** para todos los objetos? ¿Ocurre en todos los lenguajes? ¿Qué ocurre en Java?

No en todos los lenguajes, pero en muchos lenguajes orientados a objetos sí existe una clase raíz común. La idea es que todo objeto comparta un “ancestro” que define comportamientos básicos.

(1)->¿hay una **clase base** para todos los objetos?

Depende del lenguaje

En muchos lenguajes sí:

Ejemplos:

JaVa

C#

Python

En otros no necesariamente:

Ejemplos:

C++ (no hay una clase raíz obligatoria)

Lenguajes con sistemas de tipos más flexibles o estructurales

(3)-> ¿Qué ocurre en Java?

En Java sí existe una clase base universal

Clase raíz: `java.lang.Object`

Todas las clases en Java, directa o indirectamente, heredan de:

```
java.lang.Object
```

Esto significa que todos los objetos comparten ciertos métodos comunes como `toString()`, `equals()`, `hashCode()`, etc.

***CLASE

En todos los lenguajes existe una única clase base universal, aunque es una idea bastante común.

En general:

Algunos lenguajes sí definen una clase raíz de la que heredan todos los objetos.

Otros no tienen una jerarquía única (por ejemplo, permiten varios tipos base o incluso modelos sin clases estrictas).

Ejemplos:

En lenguajes como Java o C#, sí existe una clase base común.

En otros como C++, no es obligatorio que todas las clases hereden de una clase raíz única.

¿Qué ocurre en Java?

En Java:

Existe una clase base universal llamada `Object`.

Todas las clases heredan directa o indirectamente de `Object`, incluso si no lo declaras explícitamente.

Esto significa que todos los objetos comparten ciertos métodos comunes como toString(), equals(), hashCode(), etc.

***Clase

c++ no tiene una clase base implícita

Java sí la tiene : Object

8. ¿Qué es la "herencia múltiple"? ¿Existe en Java herencia múltiple?

La herencia múltiple es una característica de algunos lenguajes orientados a objetos que permite que una clase herede de más de una clase base al mismo tiempo. Es decir, una clase puede adquirir atributos y métodos de varias “clases padre”.

Ejemplo conceptual:

Una clase Híbrido podría heredar de Coche y de Barco, combinando comportamientos de ambos.

¿En qué lenguajes existe?

Sí existe en lenguajes como C++.

No todos los lenguajes la permiten directamente, porque puede generar conflictos (por ejemplo, el famoso “problema del diamante”).

***CLASE

Herencia multiple-> Heredar de mas de una clase

No existe en Java la herencia multiple

9. Las excepciones en los lenguajes orientados a objetos son objetos. Por tanto, se pueden crear excepciones personalizadas. Pon un ejemplo en Java de una excepción personalizada (UsuarioNoEncontradoException), que sea *no controlada* y que además este compuesto con un Usuario , para saber qué Usuario dio el problema. Permite además que se pueda incluir la causa, es decir, sobrecarga el

constructor para tener una versión que permita añadir la causa subyacente.

```
// Clase de dominio
public class Usuario {

    private String id;
    private String nombre;

    public Usuario(String id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }

    public String getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    @Override
    public String toString() {
        return "Usuario{id='" + id + "', nombre='" + nombre + "'}";
    }
}

// Excepción personalizada NO controlada
public class UsuarioNoEncontradoException extends RuntimeException {

    private Usuario usuario;

    // Constructor sin causa
    public UsuarioNoEncontradoException(Usuario usuario) {
        super("Usuario no encontrado: " + usuario);
        this.usuario = usuario;
    }

    // Constructor con causa subyacente
    public UsuarioNoEncontradoException(Usuario usuario, Throwable cause) {
        super("Usuario no encontrado: " + usuario, cause);
    }
}
```

```

        this.usuario = usuario;
    }

    public Usuario getUsuario() {
        return usuario;
    }
}

// Excepción personalizada NO controlada
public class UsuarioNoEncontradoException extends RuntimeException {

    private Usuario usuario;

    // Constructor sin causa
    public UsuarioNoEncontradoException(Usuario usuario) {
        super("Usuario no encontrado: " + usuario);
        this.usuario = usuario;
    }

    // Constructor con causa subyacente
    public UsuarioNoEncontradoException(Usuario usuario, Throwable cause) {
        super("Usuario no encontrado: " + usuario, cause);
        this.usuario = usuario;
    }

    public Usuario getUsuario() {
        return usuario;
    }
}

public class Main {

    public static void main(String[] args) {

        Usuario usuario = new Usuario("123", "Ana");

        try {
            buscarUsuario(usuario);
        } catch (UsuarioNoEncontradoException e) {

            System.out.println(e.getMessage());

            // Acceso al objeto Usuario que causó la excepción
            System.out.println("Usuario problemático: " + e.getUsuario());

```

```

        // Acceso a la causa
        if (e.getCause() != null) {
            System.out.println("Causa original: " + e.getCause());
        }
    }
}

public static void buscarUsuario(Usuario usuario) {

    try {
        // Simulación de error interno
        throw new IllegalArgumentException("Error en la consulta a BD");

    } catch (IllegalArgumentException e) {

        // Lanzamos nuestra excepción personalizada
        throw new UsuarioNoEncontradoException(usuario, e);
    }
}
}

```

Aspectos importantes:

Al extender RuntimeException, la excepción es no controlada (unchecked exception).

La composición se consigue mediante el atributo:

```
private Usuario usuario;
```

La causa subyacente se pasa al constructor de la superclase mediante:

```
super(mensaje, cause);
```

***CLASE

```

public class UsuarioNoEncontradoException extends RuntimeException{
    public UsuarioNoEncontradoException (String mensaje, Throwable causa, Usuario usuarioNoencontrado)
        super (mensaje, causa);
    this.usuarioNoEncontrado = usuarioNoEncontrado;
}

public UsuarioNoEncontradoException (String mensaje, Usuario usuarioNoencontrado){
    super (mensaje);
    this.usuarioNoEncontrado = usuarioNoEncontrado;
}

public Usuario getUsuarioNoEncontrado(){
    return this.usuarioNoEncontrado;
}
}

/* Usuario UsuarioABuscar=...

//throw new

```

10. Herencia vs. Composición. Se dice que no se debe emplear herencia simplemente por reutilizar código, es decir, que si quiero reutilizar código simplemente, no debo pensar en herencia como primera opción ¿por qué?

La idea de fondo es correcta: la herencia no debería usarse solo para reutilizar código.

(*)-> Diferencia clave

Herencia (is-a)

Una clase hija es un tipo de la clase padre.

Ejemplo: Perro es un Animal.

Composición (has-a)

Una clase contiene otra para usar su funcionalidad.

Ejemplo: Coche tiene un Motor.

(*)-> ¿Por qué no usar herencia solo para reutilizar código?

Porque la herencia implica algo más fuerte que “copiar comportamiento”:

Crea una relación conceptual rígida (jerarquía).

Acopla fuertemente las clases (cambios en la base afectan a las hijas).

Puede romper el diseño si la relación “es-un” no es real.

Puede violar el principio de sustitución (Liskov) si se usa mal.

(*)->Ejemplo problemático:

Hacer que Empleado herede de ArchivoPDF solo para reutilizar un método de impresión. No tiene sentido conceptual.

Cuando una subclase Hereda de otra:

```
class Animal {  
    public void respirar() { }  
}  
  
class Coche extends Animal {  
}
```

Aquí el problema no es técnico, sino de diseño:

Un Coche NO es un Animal.

Se heredó únicamente para reutilizar respirar(), pero conceptualmente la relación es incorrecta.

Esto produce varios problemas.

->Acoplamiento fuerte

La subclase depende fuertemente de la implementación de la superclase.

Si modificas la clase padre:

```
class Animal {  
    protected int edad;  
}
```

puedes romper clases hijas sin querer.

La herencia crea dependencia interna entre clases.

Con composición el acoplamiento es menor:

```

class Motor {
    public void arrancar() { }
}

class Coche {

    private Motor motor;

    public Coche(Motor motor) {
        this.motor = motor;
    }
}

```

Aquí Coche usa Motor, pero no depende de su estructura interna.

- >Rompe el encapsulamiento
- >Jerarquías rígidas
- >La herencia propaga errores de diseño
- >La composición permite cambiar comportamiento dinámicamente

Cuándo usar cada una??

Usa herencia cuando:

Existe una relación clara de tipo (“es-un”).

Quieres especializar comportamiento.

La jerarquía tiene sentido semántico.

Usa composición cuando:

Solo quieres reutilizar funcionalidad.

Necesitas flexibilidad (cambiar comportamientos en tiempo de ejecución).

No hay una relación “es-un”.

(*)-> En Java

Esto es especialmente importante porque:

Solo hay herencia simple de clases.

Se fomenta el uso de interfaces + composición.

Ejemplo típico con composición:

```

class Motor {
    void arrancar() { }
}

class Coche {
    private Motor motor = new Motor(); // composición

    void arrancar() {
        motor.arrancar();
    }
}

```

***CLASE

->No usar la Herencia solo por reutilizar el código

Debe usarse cuando se necesita la compatibilidad de tipos

->Usar herencia implica un fuerte acoplamiento desde la clase derivada hacia la clase base

-> La clase derivada depende mucho de la base

-> Cambios internos en la clase base base podrían llegar a afectar a las derivadas

11. Herencia vs. Composición. Se dice que se debe *"favorecer la composición frente a la herencia"*, ¿por qué?

La recomendación de "favorecer la composición frente a la herencia" no significa "no usar herencia", sino usar composición por defecto y herencia solo cuando encaja de verdad.

¿Por qué se prefiere la composición?

1. Menor acoplamiento

Con herencia, la clase hija queda muy ligada a la clase padre (cualquier cambio puede romperla).

Con composición, solo dependes de lo que usas, no de toda la jerarquía.

2. Más flexibilidad

La composición permite cambiar comportamientos en tiempo de ejecución:

-Puedes sustituir componentes.

-Puedes combinar funcionalidades de distintas formas.

3. Evita jerarquías artificiales

La herencia obliga a una relación "es-un".

Si esa relación no es totalmente cierta, el diseño se vuelve confuso y frágil.

4. Reduce problemas clásicos de la herencia

5. Reutilización más controlada

Con composición reutilizas exactamente lo que necesitas, no todo lo que heredas.

Ejemplo intuitivo

Herencia (menos flexible):

```
class Ave {  
    void volar() { }  
}  
  
class Pinguino extends Ave { // Problema conceptual  
    // no debería volar  
}
```

Composición (más flexible):

```
class ComportamientoVuelo {  
    void volar() { }  
}  
  
class Ave {  
    private ComportamientoVuelo vuelo;  
    public Ave(ComportamientoVuelo comportamientoVuelo) {  
        this.comportamientoVuelo = comportamientoVuelo;  
    }  
}
```

(*)->En Java

Esto cobra aún más sentido porque:

-Solo hay herencia simple de clases.

-Se promueve el uso de interfaces + composición para construir sistemas más modulares.

12. Herencia vs. Composición. Se dice que la *"herencia rompe la encapsulación"*, ¿a qué se refiere esto?

Cuando se dice que "la herencia rompe la encapsulación", se refiere a que una clase hija queda demasiado expuesta a los detalles internos de la clase padre, perdiendo el aislamiento que debería proporcionar la encapsulación.

(*)->¿Qué es la encapsulación?

La encapsulación implica que:

Una clase oculta su implementación interna.

Solo expone una interfaz pública bien definida.

El resto del código no depende de cómo está hecha por dentro.

Es decir:

Otras clases deberían depender de qué hace el objeto, no de cómo lo hace internamente.

(*)->¿Qué ocurre con la herencia?

Con herencia, la clase hija:

Depende de la implementación interna de la clase padre (no solo de su interfaz).

Puede acceder a elementos protected o comportamientos internos.

Puede verse afectada por cambios internos del padre, incluso si su interfaz pública no cambia.

Es decir, la clase padre deja de estar completamente "encapsulada".

(*)->Ejemplo típico

En Java:

```

class Lista {
    protected int tamaño;

    void añadirElemento() {
        tamaño++;
    }
}

class ListaControlada extends Lista {
    void resetear() {
        tamaño = 0; // accede directamente al estado interno
    }
}

```

Aquí:

ListaControlada depende directamente de cómo Lista gestiona su estado.
Si la implementación de Lista cambia, la clase hija puede romperse.

(*)->Problema clave

La herencia crea una relación en la que:

La clase hija no solo usa la clase padre.

Conoce y depende de sus detalles internos.

Esto rompe el principio de:

“una clase debería poder cambiar su implementación sin afectar a otras”.

El problema profundo es este:

La subclase depende del comportamiento interno implícito de la superclase.
Incluso aunque no acceda directamente a atributos.

Ejemplo clásico:

```

class Stack extends Vector {
    ...
}

```

Si Vector cambia internamente cómo gestiona elementos o sincronización, Stack puede comportarse distinto aunque la API pública de Vector siga igual.

(*)->¿Cómo lo evita la composición?

Con composición:

Solo interactúas con la interfaz pública del objeto contenido.

No accedes a su estado interno.

El acoplamiento es mucho menor.

CONCLUSION

Decir que “la herencia rompe la encapsulación” significa que:

La implementación interna deja de estar realmente oculta.

Las clases hijas quedan fuertemente acopladas a los detalles del padre.

Esto hace el sistema más frágil ante cambios.

13. Pongamos un ejemplo de dos alternativas para lo mismo. Tenemos un Estudiante y un Trabajador , ambos tienen datos en común: el DNI y el nombre. Modelemos esto de dos formas: uno por herencia, con una superclase Persona , y otro con composición, con una clase DatosPersonales . Se debe recibir una instancia de DatosPersonales en el constructor de la clase Estudiante y Trabajador .

```

class Persona {
    protected String dni;    //con private se preserva mejor la encapsulacion
    protected String nombre;

    public Persona(String dni, String nombre) {
        this.dni = dni;
        this.nombre = nombre;
    }

    public String getDni() { return dni; }
    public String getNombre() { return nombre; }
}

class Estudiante extends Persona {
    private String carrera;

    public Estudiante(String dni, String nombre, String carrera) {
        super(dni, nombre);
        this.carrera = carrera;
    }
}

class Trabajador extends Persona {
    private String empresa;

    public Trabajador(String dni, String nombre, String empresa) {
        super(dni, nombre);
        this.empresa = empresa;
    }
}

```

```

class DatosPersonales {
    private String dni;
    private String nombre;

    public DatosPersonales(String dni, String nombre) {
        this.dni = dni;
        this.nombre = nombre;
    }

    public String getDni() { return dni; }
    public String getNombre() { return nombre; }
}

class Estudiante {
    private DatosPersonales datos;
    private String carrera;

    public Estudiante(DatosPersonales datos, String carrera) {
        this.datos = datos;
        this.carrera = carrera;
    }
}

class Trabajador {
    private DatosPersonales datos;
    private String empresa;

    public Trabajador(DatosPersonales datos, String empresa) {
        this.datos = datos;
        this.empresa = empresa;
    }
}

```

Ventajas y desventajas de cada enfoque

HERENCIA:

Ventajas

- reutilización directa,
- polimorfismo,
- modelo más natural si realmente hay relación “es-un”.

Desventajas

- fuerte acoplamiento,

- rompe parcialmente encapsulación,
- jerarquías rígidas,
- más fragilidad ante cambios.

COMPOSICIÓN:

Ventajas

- menor acoplamiento,
- mejor encapsulación,
- mayor flexibilidad,
- reutilización más segura.

Desventajas

- más delegación,
- algo más de código,
- no hay polimorfismo automático.

Aquí:

Estudiante tiene datos personales

Trabajador tiene datos personales

()->Diferencia conceptual clave

Herencia → relación “es-un”

(Estudiante es una persona)

Composición → relación “tiene-un”

(Estudiante tiene datos personales)